Process & Decision Documentation

Side Quests

A 2D pixel game where play as a fox racing against the clock. The objective is to navigate a multi-tiered tile-based level to find a randomly spawning treasure chest before the 30-second timer runs out.

If GenAI was used (keep each response as brief as possible):

Date Used: Mar 8, 2026

Tool Disclosure: Gemini 3.1 Pro

Purpose of Use: Code generation, adding dynamic audio states, implementing a game state machine (win/lose/timer), and HTML/CSS DOM styling for a p5.js sketch.

Summary of Interaction: Added sound effects, a movement-triggered timer, randomized item spawns, Win/Lose states, and debugged text rendering and physics collider scaling issues.

Human Decision Point(s): Upload sound effects and images, design the game's overall mechanics, iterate on game details, and playtest the entire game.

Integrity & Verification Note: All code was integrated and play tested. Audio state logic was manually tested to prevent overlapping loops, and hitboxes were tested to ensure accurate collision with the treasure.

Scope of GenAI Use: Implementation of game logic (timers, states, randomized arrays), audio and sound effect, and UI styling.

Limitations or Misfires: The AI initially attempted to apply standard web fonts (Arial/Calibri) directly to the low-resolution, anti-aliased canvas, resulting in unreadable text. Additionally, when scaling the custom image asset, the AI resized the visual texture but did not initially adjust the underlying p5play physics collider, leading to an bug.

Summary of Process (Human + Tool)

I started with add environmental audio (running, jumping, falling), ensuring the running sound looped conditionally based on player movement and ground contact. Next, I built the core game loop: a 30-second countdown that only starts upon initial input, paired with

a randomly spawning treasure object. Finally, I added game-ending states (Win/Lose) and spent time debugging the game visual, ultimately fix all the issues.

Decision Points & Trade-offs

I searched online for sound effects, ran tests, replaced several options, and finally selected the three most suitable ones.

I wanted a clean serif font (Times New Roman) for the UI. However, rendering this directly on a 240x192 canvas with noSmooth() turned on completely garbled the text. I decided to overlay HTML elements on top of the canvas. This allowing the text to be clearly.

Verification & Judgement

During playtesting, I noticed the fox was triggering the "Win" state before actually touching the treasure. I used this observation to deduce that p5play's image.resize() function only scales the visual representation, not the physics body. I verified that manually overriding the sprite's collider (treasure.w = 16; treasure.h = 16;) fixed the discrepancy.

I also tested the game's mechanics, sound effects, movement, countdown, and other elements to see if it could deliver a smooth gaming experience.

Limitations, Dead Ends, or Open Questions

Limitation: The p5play library's handling of imported image sizes requires manual synchronization between the visual texture size and the rigid-body physics dimensions.

Appendix

Appendix: GenAI Transcript

Tool Used: Gemini **Date:** Mar 8, 2026

User: Based on the code files I uploaded, add three sound effects for the fox. When the fox runs, use: running.mp3 When jumping to a platform, use: jumping.mp3 After falling, use: falling.mp3 I have added all the sound effects to the Assets folder. Give me the new code.

Gemini: Here is the updated code to get your fox sounding as good as it looks!

To make this work, **you must also update your `index.html` file**. The standard p5.js library doesn't include sound capabilities by default, so we need to add the `p5.sound` add-on script.

1. Update `index.html`

Add the `p5.sound.min.js` script right below your main p5.js script.

HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>GBDA302 Week 6 - Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background</title>
    <style>
      body {
        margin: 0;
        overflow: hidden;
        background: #222;
      }
      canvas {
        display: block;
      }
    </style>
  </head>
  <body>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script>

    <script src="https://p5play.org/v3/planck.min.js"></script>
    <script src="https://p5play.org/v3/p5play.min.js"></script>
    <script src="sketch.js"></script>
  </body>
</html>
```

---

2. Update `sketch.js`

I have added the variables at the top, loaded the sounds in `preload()`, and added the audio logic directly to your jumping, movement, and falling states.

(Note: The running sound uses a `.loop()` method so it plays continuously while the left or right keys are held down on the ground, and `.stop()` when the player stops or jumps).

JavaScript

```javascript
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026
```

```
  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false; // track if the player is attacking
let attackFrameCounter = 0; // tracking attack animation

// --- TILE MAP ---
let level = [
  "                  g                  ", // row  0
  "                                     ", // row  1
  "              LggR                   ", // row  2
  "     LR   LgR         LR             ", // row  3: upper platforms
  "                                     ", // row  4
  "   LgggR      LR   LgR               ", // row  5: mid platforms
  "        LgR           g   LggggR     ", // row  6: walls + low platform
  "             LgR                     ", // row  7
  "                              LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
  "         [d]        gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];
```

```
// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  // --- IMAGES ---
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  // --- AUDIO ---
  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";
```

```
groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundTile2Img;
groundDeep.tile = "d";

platformsL = new Group();
platformsL.physics = "static";
platformsL.img = platformTileLImg;
platformsL.tile = "L";

platformsR = new Group();
platformsR.physics = "static";
platformsR.img = platformTileRImg;
platformsR.tile = "R";

wallsL = new Group();
wallsL.physics = "static";
wallsL.img = wallTileLImg;
wallsL.tile = "[";

wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- GROUND SENSOR ---
sensor = new Sprite();
sensor.x = player.x;
sensor.y = player.y + player.h / 2;
sensor.w = player.w;
sensor.h = 2;
sensor.mass = 0.01;
sensor.removeColliders();
sensor.visible = false;
let sensorJoint = new GlueJoint(player, sensor);
sensorJoint.visible = false;
```

```
  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);
  world.step();

  // --- CAMERA ---
  camera.width = VIEWW;
  camera.height = VIEWH;

  let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
  let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

  camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
  camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

  // --- PLAYER CONTROLS ---
  let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play();
  }

  // -- JUMP --
  if (grounded && kb.presses("up")) {
    player.vel.y = -4.5;
    jumpSound.play(); // Play jump sound effect
  }

  // --- STATE MACHINE ---
  if (attacking) {
    attackFrameCounter++;
    if (attackFrameCounter > 12) {
      attacking = false;
      attackFrameCounter = 0;
    }
  } else if (!grounded) {
```

```
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop(); // Loop running sound if moving on the ground
  }
} else {
  if (runSound.isPlaying()) {
    runSound.stop(); // Stop sound if in the air, stopped, or attacking
  }
}

// --- PLAYER BOUNDS ---
player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PLAYER LOSE STATE ---
```

```
  if (player.y > LEVELH + TILE_H * 3) {
    player.x = FRAME_W;
    player.y = PLAYER_START_Y;
    fallSound.play(); // Play falling sound effect on reset
  }

  // --- PIXEL SNAP ---
  const px = player.x, py = player.y;
  const sx = sensor.x, sy = sensor.y;

  player.x = Math.round(player.x);
  player.y = Math.round(player.y);
  sensor.x = Math.round(sensor.x);
  sensor.y = Math.round(sensor.y);

  allSprites.draw();

  player.x = px;
  player.y = py;
  sensor.x = sx;
  sensor.y = sy;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to help you add an attack sound to the space bar input as well?

User: Add a 30-second countdown timer at the top with the text “Find the treasure before time runs out.” The countdown only starts ticking once the fox begins moving. The treasure's location is randomly refreshed each time, but always remains on the upper levels.

Gemini: This sounds like a fun addition! To make this work, we need to introduce a few new concepts: a timer variable, a state to check if the player has started moving, an array of specific coordinates (spawn points) on the upper levels, and a new Sprite for the treasure.

Since you didn't mention having a specific image for the treasure, I have created it as a simple **gold square** using p5play's built-in drawing features. You can easily swap this out for an image later using `treasure.img = loadImage(...)` just like you did with the tiles.
Here is your updated `sketch.js` file:
JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling
Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 }, // Row 2 LggR
  { c: 5,  r: 2 }, // Row 3 LR (Left)
  { c: 11, r: 2 }, // Row 3 LgR (Middle)
  { c: 23, r: 2 }, // Row 3 LR (Right)
```

```
  { c: 4,  r: 4 }, // Row 5 LgggR
  { c: 15, r: 4 }, // Row 5 LR
  { c: 21, r: 4 }  // Row 5 LgR
];

// --- TILE MAP ---
let level = [
  "                g                  ", // row  0
  "                                   ", // row  1
  "            LggR                   ", // row  2
  "    LR   LgR         LR            ", // row  3: upper platforms
  "                                   ", // row  4
  "   LgggR      LR   LgR             ", // row  5: mid platforms
  "       LgR          g   LggggR     ", // row  6: walls + low platform
  "           LgR                     ", // row  7
  "                            LggR", // row  8
  "        LgR             LR  LR  [dd]", // row  9: mid-right platform
  "        [d]       gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  ddddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  runSound = loadSound("assets/running.mp3");
```

```
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
  groundDeep.tile = "d";

  platformsL = new Group();
  platformsL.physics = "static";
  platformsL.img = platformTileLImg;
  platformsL.tile = "L";

  platformsR = new Group();
  platformsR.physics = "static";
  platformsR.img = platformTileRImg;
  platformsR.tile = "R";

  wallsL = new Group();
  wallsL.physics = "static";
  wallsL.img = wallTileLImg;
  wallsL.tile = "[";

  wallsR = new Group();
  wallsR.physics = "static";
  wallsR.img = wallTileRImg;
  wallsR.tile = "]";

  new Tiles(level, 0, 0, TILE_W, TILE_H);

  // --- PLAYER ---
  player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
  player.spriteSheet = playerImg;
  player.rotationLock = true;
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
```

```
  player.anis.offset.y = -8;
  player.addAnis(playerAnis);

  player.ani = "idle";
  player.w = 18;
  player.h = 12;
  player.friction = 0;
  player.bounciness = 0;

  // --- TREASURE ---
  treasure = new Sprite();
  treasure.w = 12;
  treasure.h = 12;
  treasure.color = "gold"; // Simple placeholder color
  treasure.collider = "static";
  // Create an overlap sensor rather than a physical wall
  treasure.overlaps(player);

  // Initialize the level and place the first treasure
  resetLevel(false);

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);
  world.step();

  // --- TIMER LOGIC ---
  if (!timerStarted) {
    // Only start the timer if the player presses a movement key
    if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
      timerStarted = true;
      lastTick = millis();
```

```
    }
  } else {
    // Decrease the timer by 1 every 1000 milliseconds (1 second)
    if (millis() - lastTick >= 1000 && timeLeft > 0) {
      timeLeft--;
      lastTick = millis();
    }
  }

  // --- CAMERA ---
  camera.width = VIEWW;
  camera.height = VIEWH;

  let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
  let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

  camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
  camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

  // --- PLAYER CONTROLS ---
  let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play();
  }

  // -- JUMP --
  if (grounded && kb.presses("up")) {
    player.vel.y = -4.5;
    jumpSound.play();
  }

  // --- STATE MACHINE ---
  if (attacking) {
    attackFrameCounter++;
    if (attackFrameCounter > 12) {
      attacking = false;
      attackFrameCounter = 0;
    }
  } else if (!grounded) {
    player.ani = "jump";
    player.ani.frame = player.vel.y < 0 ? 0 : 1;
  } else {
    player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
  }
```

```
// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop();
  }
} else {
  if (runSound.isPlaying()) {
    runSound.stop();
  }
}

// --- PLAYER BOUNDS ---
player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- GAME END/RESET STATES ---
// If player touches treasure, time runs out, or player falls off map
if (player.overlaps(treasure) || timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
  // If they fell off the map, play the fall sound
  let fell = player.y > LEVELH + TILE_H * 3;
  resetLevel(fell);
}
```

```
  // --- PIXEL SNAP ---
  const px = player.x, py = player.y;
  const sx = sensor.x, sy = sensor.y;

  player.x = Math.round(player.x);
  player.y = Math.round(player.y);
  sensor.x = Math.round(sensor.x);
  sensor.y = Math.round(sensor.y);

  allSprites.draw();

  player.x = px;
  player.y = py;
  sensor.x = sx;
  sensor.y = sy;

  // --- UI RENDERING ---
  // We draw the UI last so it sits on top of all the sprites and backgrounds
  camera.off();

  // Semi-transparent background banner for readability
  fill(0, 150);
  noStroke();
  rect(0, 0, VIEWW, 16);

  // Timer Text
  fill(255);
  textAlign(CENTER, TOP);
  textSize(8); // Keep this small due to the 240px wide pixelated canvas
  text(`Find the treasure before time runs out: ${timeLeft}s`, VIEWW / 2, 4);

  camera.on();
}

// Resets the player, timer, and randomizes the treasure spawn
function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  // Reset timer
  timeLeft = 30;
  timerStarted = false;

  // Randomize treasure location from spawn points
  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
```

```
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to show you how to swap the golden square for an image of a real treasure chest next?

User: After the game ends, a screen appears displaying “YOU WIN” (green) or “YOU LOSE” (red) along with “Press R to restart.”

Gemini: This is a great way to add a complete gameplay loop! We can achieve this by introducing a **"state machine"** for the game itself. We'll use a new variable called `gameState` to track whether the game is actively playing, won, or lost.

When the game enters the "WIN" or "LOSE" state, we will stop updating the physics and player controls (freezing the screen) and draw the overlay text instead. Pressing "R" will then trigger the `resetLevel()` function to start things over.

Here is the updated `sketch.js` code with the new win/lose logic and screen:

JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling
Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor, treasure;
```

```
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING"; // Tracks if we are 'PLAYING', 'WIN', or 'LOSE'
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 }, // Row 2 LggR
  { c: 5,  r: 2 }, // Row 3 LR (Left)
  { c: 11, r: 2 }, // Row 3 LgR (Middle)
  { c: 23, r: 2 }, // Row 3 LR (Right)
  { c: 4,  r: 4 }, // Row 5 LgggR
  { c: 15, r: 4 }, // Row 5 LR
  { c: 21, r: 4 }  // Row 5 LgR
];

// --- TILE MAP ---
let level = [
  "                g                  ", // row  0
  "                                   ", // row  1
  "            LggR                   ", // row  2
  "    LR   LgR        LR             ", // row  3: upper platforms
  "                                   ", // row  4
  "  LgggR      LR   LgR              ", // row  5: mid platforms
  "       LgR          g   LggggR     ", // row  6: walls + low platform
  "           LgR                     ", // row  7
  "                            LggR", // row  8
  "        LgR             LR  LR  [dd]", // row  9: mid-right platform
  "        [d]       gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];
```

```
// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
```

```
groundDeep.img = groundTile2Img;
groundDeep.tile = "d";

platformsL = new Group();
platformsL.physics = "static";
platformsL.img = platformTileLImg;
platformsL.tile = "L";

platformsR = new Group();
platformsR.physics = "static";
platformsR.img = platformTileRImg;
platformsR.tile = "R";

wallsL = new Group();
wallsL.physics = "static";
wallsL.img = wallTileLImg;
wallsL.tile = "[";

wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- TREASURE ---
treasure = new Sprite();
treasure.w = 12;
treasure.h = 12;
treasure.color = "gold";
treasure.collider = "static";
treasure.overlaps(player);

resetLevel(false);

// --- GROUND SENSOR ---
sensor = new Sprite();
sensor.x = player.x;
```

```
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  // Only update game mechanics if we are actively playing
  if (gameState === "PLAYING") {
    world.step();

    // --- TIMER LOGIC ---
    if (!timerStarted) {
      if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
        timerStarted = true;
        lastTick = millis();
      }
    } else {
      if (millis() - lastTick >= 1000 && timeLeft > 0) {
        timeLeft--;
        lastTick = millis();
      }
    }

    // --- CAMERA ---
    camera.width = VIEWW;
    camera.height = VIEWH;

    let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
    let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

    camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
    camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

    // --- PLAYER CONTROLS ---
    let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);
```

```
// -- ATTACK INPUT --
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4.5;
  jumpSound.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop();
  }
} else {
  if (runSound.isPlaying()) {
    runSound.stop();
```

```
    }
  }

  // --- PLAYER BOUNDS ---
  player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

  // --- GAME END/WIN STATES ---
  if (player.overlaps(treasure)) {
    gameState = "WIN";
    if (runSound.isPlaying()) runSound.stop();
  } else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
    if (player.y > LEVELH + TILE_H * 3) {
      fallSound.play();
    }
    gameState = "LOSE";
    if (runSound.isPlaying()) runSound.stop();
  }
}

// --- DRAWING SECTION ---
// The code below draws every frame regardless of whether the game is paused/ended,
// ensuring the backgrounds and sprites stay visible behind the end screen.

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PIXEL SNAP ---
const px = player.x, py = player.y;
const sx = sensor.x, sy = sensor.y;

player.x = Math.round(player.x);
player.y = Math.round(player.y);
sensor.x = Math.round(sensor.x);
sensor.y = Math.round(sensor.y);

allSprites.draw();

player.x = px;
player.y = py;
```

```
  sensor.x = sx;
  sensor.y = sy;

  // --- UI RENDERING ---
  camera.off();

  if (gameState === "PLAYING") {
    // Normal HUD
    fill(0, 150);
    noStroke();
    rect(0, 0, VIEWW, 16);

    fill(255);
    textAlign(CENTER, TOP);
    textSize(8);
    text(`Find the treasure before time runs out: ${timeLeft}s`, VIEWW / 2, 4);
  } else {
    // End Game Overlay
    fill(0, 180); // Dark, semi-transparent backdrop
    rect(0, 0, VIEWW, VIEWH);

    textAlign(CENTER, CENTER);
    if (gameState === "WIN") {
      fill(0, 255, 0); // Green
      textSize(16);
      text("YOU WIN", VIEWW / 2, VIEWH / 2 - 10);
    } else if (gameState === "LOSE") {
      fill(255, 0, 0); // Red
      textSize(16);
      text("YOU LOSE", VIEWW / 2, VIEWH / 2 - 10);
    }

    fill(255);
    textSize(8);
    text("Press R to restart", VIEWW / 2, VIEWH / 2 + 10);

    // Press R to Restart
    if (kb.presses("r")) {
      resetLevel(false);
    }
  }

  camera.on();
}

// Resets the player, timer, state, and randomizes the treasure spawn
function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
```

```
    fallSound.play();
  }

  // Reset states
  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  // Randomize treasure location from spawn points
  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to help you add a fun win sound effect to play when the treasure is found?

User: The current font is a bit hard to read. Please use Calibri or Arial. The countdown "30s" and "Find the treasure before time runs out" should be on two separate lines.

Gemini: This is an easy fix! To make the text more readable and split it across two lines, we just need to set the textFont() before we draw the text and adjust our text drawing coordinates.
I also increased the height of the semi-transparent background box slightly so both lines fit comfortably inside it.

Here is the updated sketch.js code:
JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling
Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
```

```
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5,  r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4,  r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 }
];

// --- TILE MAP ---
let level = [
  "                g                  ", // row  0
  "                                   ", // row  1
  "            LggR                   ", // row  2
  "     LR   LgR         LR           ", // row  3: upper platforms
  "                                   ", // row  4
```

```
  "   LgggR       LR   LgR                ", // row  5: mid platforms
  "        LgR           g   LggggR      ", // row  6: walls + low platform
  "             LgR                    ", // row  7
  "                              LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
  "         [d]        gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
```

```
world.gravity.y = GRAVITY;

// --- TILE GROUPS ---
ground = new Group();
ground.physics = "static";
ground.img = groundTile1Img;
ground.tile = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundTile2Img;
groundDeep.tile = "d";

platformsL = new Group();
platformsL.physics = "static";
platformsL.img = platformTileLImg;
platformsL.tile = "L";

platformsR = new Group();
platformsR.physics = "static";
platformsR.img = platformTileRImg;
platformsR.tile = "R";

wallsL = new Group();
wallsL.physics = "static";
wallsL.img = wallTileLImg;
wallsL.tile = "[";

wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- TREASURE ---
treasure = new Sprite();
treasure.w = 12;
```

```
  treasure.h = 12;
  treasure.color = "gold";
  treasure.collider = "static";
  treasure.overlaps(player);

  resetLevel(false);

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  if (gameState === "PLAYING") {
    world.step();

    // --- TIMER LOGIC ---
    if (!timerStarted) {
      if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
        timerStarted = true;
        lastTick = millis();
      }
    } else {
      if (millis() - lastTick >= 1000 && timeLeft > 0) {
        timeLeft--;
        lastTick = millis();
      }
    }

    // --- CAMERA ---
    camera.width = VIEWW;
    camera.height = VIEWH;

    let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
```

```
    let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

    camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
    camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

    // --- PLAYER CONTROLS ---
    let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);

    // -- ATTACK INPUT --
    if (grounded && !attacking && kb.presses("space")) {
      attacking = true;
      attackFrameCounter = 0;
      player.vel.x = 0;
      player.ani.frame = 0;
      player.ani = "attack";
      player.ani.play();
    }

    // -- JUMP --
    if (grounded && kb.presses("up")) {
      player.vel.y = -4.5;
      jumpSound.play();
    }

    // --- STATE MACHINE ---
    if (attacking) {
      attackFrameCounter++;
      if (attackFrameCounter > 12) {
        attacking = false;
        attackFrameCounter = 0;
      }
    } else if (!grounded) {
      player.ani = "jump";
      player.ani.frame = player.vel.y < 0 ? 0 : 1;
    } else {
      player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
    }

    // --- MOVEMENT ---
    let isMoving = false;
    if (!attacking) {
      player.vel.x = 0;
      if (kb.pressing("left")) {
        player.vel.x = -1.5;
        player.mirror.x = true;
        isMoving = true;
      } else if (kb.pressing("right")) {
        player.vel.x = 1.5;
        player.mirror.x = false;
        isMoving = true;
      }
    }
```

```
  // --- RUNNING AUDIO LOGIC ---
  if (grounded && isMoving && !attacking) {
    if (!runSound.isPlaying()) {
      runSound.loop();
    }
  } else {
    if (runSound.isPlaying()) {
      runSound.stop();
    }
  }

  // --- PLAYER BOUNDS ---
  player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

  // --- GAME END/WIN STATES ---
  if (player.overlaps(treasure)) {
    gameState = "WIN";
    if (runSound.isPlaying()) runSound.stop();
  } else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
    if (player.y > LEVELH + TILE_H * 3) {
      fallSound.play();
    }
    gameState = "LOSE";
    if (runSound.isPlaying()) runSound.stop();
  }
}

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PIXEL SNAP ---
const px = player.x, py = player.y;
const sx = sensor.x, sy = sensor.y;

player.x = Math.round(player.x);
player.y = Math.round(player.y);
sensor.x = Math.round(sensor.x);
sensor.y = Math.round(sensor.y);
```

```
allSprites.draw();

player.x = px;
player.y = py;
sensor.x = sx;
sensor.y = sy;

// --- UI RENDERING ---
camera.off();

// Apply a cleaner font
textFont('Calibri, Arial, sans-serif');

if (gameState === "PLAYING") {
  // Normal HUD
  fill(0, 150);
  noStroke();
  // Made the background rect taller to fit two lines
  rect(0, 0, VIEWW, 26);

  fill(255);
  textAlign(CENTER, TOP);
  textSize(8);

  // Split into two lines
  text("Find the treasure before time runs out", VIEWW / 2, 4);
  text(`${timeLeft}s`, VIEWW / 2, 14);
} else {
  // End Game Overlay
  fill(0, 180);
  rect(0, 0, VIEWW, VIEWH);

  textAlign(CENTER, CENTER);
  if (gameState === "WIN") {
    fill(0, 255, 0);
    textSize(16);
    text("YOU WIN", VIEWW / 2, VIEWH / 2 - 10);
  } else if (gameState === "LOSE") {
    fill(255, 0, 0);
    textSize(16);
    text("YOU LOSE", VIEWW / 2, VIEWH / 2 - 10);
  }

  fill(255);
  textSize(8);
  text("Press R to restart", VIEWW / 2, VIEWH / 2 + 10);

  if (kb.presses("r")) {
    resetLevel(false);
  }
}
```

```
  camera.on();
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to add a simple flashing effect or a color change to the countdown timer when it gets down to the last 5 seconds to build up some tension?

User: Still not working, Can you give me a suitable font?

Gemini: Ah, I see exactly what is happening! Because your game canvas is very small (240x192 pixels) and we turned off anti-aliasing to keep the pixel art looking crisp (noSmooth()), standard web fonts like Calibri or Arial get completely mangled. They rely on smooth curves that just don't exist on a tiny pixel grid.

To fix this, we need to use a dedicated **pixel font**. A great, readable choice for this is **"VT323"** from Google Fonts.

Here is how to add it so your text looks sharp and fits the retro style perfectly.

1. Update your index.html

We need to add a link to the Google Font in the <head> of your HTML document so the game knows where to find it.

HTML

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>GBDA302 Week 6 - Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background</title>

    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=VT323&display=swap" rel="stylesheet">

    <style>
      body {
        margin: 0;
        overflow: hidden;
        background: #222;
      }
      canvas {
        display: block;
      }
    </style>
  </head>
  <body>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script>

    <script src="https://p5play.org/v3/planck.min.js"></script>
    <script src="https://p5play.org/v3/p5play.min.js"></script>
    <script src="sketch.js"></script>
  </body>
</html>
```

---

2. Update your `sketch.js`

I have updated the `textFont()` to use "VT323" and increased the `textSize()` to 12. Pixel fonts are usually naturally smaller, so bumping up the size to 12 makes it perfectly readable while still fitting beautifully on your screen. I also adjusted the background banner size slightly to comfortably fit the text.

(Just replace your entire `sketch.js` with this code)

JavaScript

```javascript
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack
```

```
  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5,  r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4,  r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 }
];

// --- TILE MAP ---
let level = [
  "                    g                    ", // row  0
  "                                         ", // row  1
  "                LggR                     ", // row  2
```

```
  "     LR   LgR          LR                ", // row  3: upper platforms
  "                                        ", // row  4
  "   LgggR       LR   LgR                 ", // row  5: mid platforms
  "        LgR           g   LggggR       ", // row  6: walls + low platform
  "             LgR                        ", // row  7
  "                                 LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
  "         [d]        gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);
```

```
allSprites.pixelPerfect = true;
world.gravity.y = GRAVITY;

// --- TILE GROUPS ---
ground = new Group();
ground.physics = "static";
ground.img = groundTile1Img;
ground.tile = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundTile2Img;
groundDeep.tile = "d";

platformsL = new Group();
platformsL.physics = "static";
platformsL.img = platformTileLImg;
platformsL.tile = "L";

platformsR = new Group();
platformsR.physics = "static";
platformsR.img = platformTileRImg;
platformsR.tile = "R";

wallsL = new Group();
wallsL.physics = "static";
wallsL.img = wallTileLImg;
wallsL.tile = "[";

wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- TREASURE ---
```

```
  treasure = new Sprite();
  treasure.w = 12;
  treasure.h = 12;
  treasure.color = "gold";
  treasure.collider = "static";
  treasure.overlaps(player);

  resetLevel(false);

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  if (gameState === "PLAYING") {
    world.step();

    // --- TIMER LOGIC ---
    if (!timerStarted) {
      if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
        timerStarted = true;
        lastTick = millis();
      }
    } else {
      if (millis() - lastTick >= 1000 && timeLeft > 0) {
        timeLeft--;
        lastTick = millis();
      }
    }

    // --- CAMERA ---
    camera.width = VIEWW;
    camera.height = VIEWH;
```

```
  let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
  let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

  camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
  camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

  // --- PLAYER CONTROLS ---
  let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play();
  }

  // -- JUMP --
  if (grounded && kb.presses("up")) {
    player.vel.y = -4.5;
    jumpSound.play();
  }

  // --- STATE MACHINE ---
  if (attacking) {
    attackFrameCounter++;
    if (attackFrameCounter > 12) {
      attacking = false;
      attackFrameCounter = 0;
    }
  } else if (!grounded) {
    player.ani = "jump";
    player.ani.frame = player.vel.y < 0 ? 0 : 1;
  } else {
    player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
  }

  // --- MOVEMENT ---
  let isMoving = false;
  if (!attacking) {
    player.vel.x = 0;
    if (kb.pressing("left")) {
      player.vel.x = -1.5;
      player.mirror.x = true;
      isMoving = true;
    } else if (kb.pressing("right")) {
      player.vel.x = 1.5;
      player.mirror.x = false;
      isMoving = true;
```

```
    }
  }

  // --- RUNNING AUDIO LOGIC ---
  if (grounded && isMoving && !attacking) {
    if (!runSound.isPlaying()) {
      runSound.loop();
    }
  } else {
    if (runSound.isPlaying()) {
      runSound.stop();
    }
  }

  // --- PLAYER BOUNDS ---
  player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

  // --- GAME END/WIN STATES ---
  if (player.overlaps(treasure)) {
    gameState = "WIN";
    if (runSound.isPlaying()) runSound.stop();
  } else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
    if (player.y > LEVELH + TILE_H * 3) {
      fallSound.play();
    }
    gameState = "LOSE";
    if (runSound.isPlaying()) runSound.stop();
  }
}

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PIXEL SNAP ---
const px = player.x, py = player.y;
const sx = sensor.x, sy = sensor.y;

player.x = Math.round(player.x);
player.y = Math.round(player.y);
```

```
sensor.x = Math.round(sensor.x);
sensor.y = Math.round(sensor.y);

allSprites.draw();

player.x = px;
player.y = py;
sensor.x = sx;
sensor.y = sy;

// --- UI RENDERING ---
camera.off();

// Apply the newly loaded VT323 pixel font
textFont('VT323');

if (gameState === "PLAYING") {
  // Normal HUD
  fill(0, 150);
  noStroke();
  // Taller background rect to fit the larger font
  rect(0, 0, VIEWW, 30);

  fill(255);
  textAlign(CENTER, TOP);
  // Increased text size from 8 to 12
  textSize(12);

  // Split into two lines
  text("Find the treasure before time runs out", VIEWW / 2, 4);
  text(`${timeLeft}s`, VIEWW / 2, 16);
} else {
  // End Game Overlay
  fill(0, 180);
  rect(0, 0, VIEWW, VIEWH);

  textAlign(CENTER, CENTER);
  if (gameState === "WIN") {
    fill(0, 255, 0);
    textSize(20);
    text("YOU WIN", VIEWW / 2, VIEWH / 2 - 12);
  } else if (gameState === "LOSE") {
    fill(255, 0, 0);
    textSize(20);
    text("YOU LOSE", VIEWW / 2, VIEWH / 2 - 12);
  }

  fill(255);
  textSize(12);
  text("Press R to restart", VIEWW / 2, VIEWH / 2 + 12);

  if (kb.presses("r")) {
    resetLevel(false);
```

```
    }
  }

  camera.on();
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to show you how to swap the treasure's plain gold square for an actual image sprite next?

User: Can you change the font to Times New Roman?

Gemini: I can certainly help you change the font to Times New Roman, but I need to be completely honest with you: **it will likely look exactly like your first screenshot (garbled and unreadable).** Here is why that is happening: Your game canvas is very small (only 240 pixels wide by 192 pixels tall) to achieve that retro, pixel-art look. We also use the `noSmooth()` function, which turns off anti-aliasing. Standard fonts like Arial, Calibri, or Times New Roman rely on anti-aliasing (smoothing) to look readable. When you force them onto a tiny, unsmoothed pixel grid, they break apart.

If you want to try it anyway inside the canvas, you can change the `textFont` line in your `sketch.js` file (down near line 380) to this:

JavaScript

```
// Apply Times New Roman font
textFont('Times New Roman');
```

How to actually fix this:

If you really want clean, crisp Times New Roman text while keeping the retro pixel-art style for your game, the best approach is to move the text **outside** of the <canvas> entirely. We can use standard HTML and CSS to float the text directly over the game. This way, the text renders at your monitor's full resolution, while the game remains pixelated.

Would you like me to rewrite the code to use an HTML overlay so your Times New Roman text looks perfectly crisp and readable?

User: I've added a new treasure image (treasure.png) to the assets folder. Please update the code for me.

Gemini: This is a perfect time to fix two birds with one stone!

I have updated your code to load the new treasure.png image. I also went ahead and solved that blurry text issue from your previous message. Instead of drawing the text onto your tiny pixelated canvas (which crushes the letters), I used p5's createDiv() to draw standard HTML text directly over the top of your game.

This means **you now have crystal-clear Times New Roman text** while keeping the retro pixel-art style of the game itself intact!

Here is your updated sketch.js file. Just replace your existing code with this:

JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/
```

```
let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg, treasureImg;

// UI variables for our HTML overlay
let uiDiv, endDiv;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5,  r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4,  r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 }
];

// --- TILE MAP ---
let level = [
  "                g                  ", // row  0
  "                                   ", // row  1
  "            LggR                   ", // row  2
  "    LR   LgR         LR            ", // row  3: upper platforms
  "                                   ", // row  4
  "  LgggR      LR   LgR              ", // row  5: mid platforms
  "       LgR           g   LggggR    ", // row  6: walls + low platform
  "           LgR                     ", // row  7
  "                            LggR", // row  8
  "        LgR              LR  LR  [dd]", // row  9: mid-right platform
  "        [d]        gggg          [dd]", // row 10: wall below mid-right platform
```

```
  "ggggg  ggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  ddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  // Load the new treasure image
  treasureImg = loadImage("assets/treasure.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- HTML UI OVERLAY (Crisp Times New Roman text!) ---
```

```
  uiDiv = createDiv('<div style="font-family: \'Times New Roman\', serif; font-size: 24px;">Find the
treasure before time runs out<br><span id="time-span">30s</span></div>');
  uiDiv.position(0, 0);
  uiDiv.style('width', '100%');
  uiDiv.style('background', 'rgba(0, 0, 0, 0.6)');
  uiDiv.style('color', 'white');
  uiDiv.style('text-align', 'center');
  uiDiv.style('padding', '10px 0');
  uiDiv.style('pointer-events', 'none');

  endDiv = createDiv('<div id="end-title" style="font-family: \'Times New Roman\', serif; font-size:
48px; font-weight: bold;"></div><div style="font-family: \'Times New Roman\', serif; font-size: 24px;
margin-top: 10px; color: white;">Press R to restart</div>');
  endDiv.position(0, 0);
  endDiv.style('width', '100%');
  endDiv.style('height', '100%');
  endDiv.style('background', 'rgba(0, 0, 0, 0.8)');
  endDiv.style('display', 'none');
  endDiv.style('flex-direction', 'column');
  endDiv.style('justify-content', 'center');
  endDiv.style('align-items', 'center');
  endDiv.style('pointer-events', 'none');

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
  groundDeep.tile = "d";

  platformsL = new Group();
  platformsL.physics = "static";
  platformsL.img = platformTileLImg;
  platformsL.tile = "L";

  platformsR = new Group();
  platformsR.physics = "static";
  platformsR.img = platformTileRImg;
  platformsR.tile = "R";

  wallsL = new Group();
  wallsL.physics = "static";
  wallsL.img = wallTileLImg;
  wallsL.tile = "[";

  wallsR = new Group();
  wallsR.physics = "static";
  wallsR.img = wallTileRImg;
  wallsR.tile = "]";
```

```
  new Tiles(level, 0, 0, TILE_W, TILE_H);

  // --- PLAYER ---
  player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
  player.spriteSheet = playerImg;
  player.rotationLock = true;
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
  player.anis.offset.y = -8;
  player.addAnis(playerAnis);

  player.ani = "idle";
  player.w = 18;
  player.h = 12;
  player.friction = 0;
  player.bounciness = 0;

  // --- TREASURE ---
  treasure = new Sprite();
  treasure.img = treasureImg; // Attach the new image
  treasure.collider = "static";
  treasure.overlaps(player);

  resetLevel(false);

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  if (gameState === "PLAYING") {
    world.step();
```

```
  // --- TIMER LOGIC ---
  if (!timerStarted) {
    if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
      timerStarted = true;
      lastTick = millis();
    }
  } else {
    if (millis() - lastTick >= 1000 && timeLeft > 0) {
      timeLeft--;
      lastTick = millis();
      // Update the HTML timer text
      document.getElementById('time-span').innerText = timeLeft + 's';
    }
  }

  // --- CAMERA ---
  camera.width = VIEWW;
  camera.height = VIEWH;

  let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
  let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

  camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
  camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

  // --- PLAYER CONTROLS ---
  let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play();
  }

  // -- JUMP --
  if (grounded && kb.presses("up")) {
    player.vel.y = -4.5;
    jumpSound.play();
  }

  // --- STATE MACHINE ---
  if (attacking) {
    attackFrameCounter++;
    if (attackFrameCounter > 12) {
      attacking = false;
      attackFrameCounter = 0;
    }
```

```
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop();
  }
} else {
  if (runSound.isPlaying()) {
    runSound.stop();
  }
}

// --- PLAYER BOUNDS ---
player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

// --- GAME END/WIN STATES ---
if (player.overlaps(treasure)) {
  gameState = "WIN";
  if (runSound.isPlaying()) runSound.stop();

  // Update HTML UI
  uiDiv.hide();
  endDiv.style('display', 'flex');
  document.getElementById('end-title').innerText = "YOU WIN";
  document.getElementById('end-title').style.color = "#00ff00";

} else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
  if (player.y > LEVELH + TILE_H * 3) {
    fallSound.play();
  }
  gameState = "LOSE";
  if (runSound.isPlaying()) runSound.stop();
```

```
      // Update HTML UI
      uiDiv.hide();
      endDiv.style('display', 'flex');
      document.getElementById('end-title').innerText = "YOU LOSE";
      document.getElementById('end-title').style.color = "#ff0000";
    }
  } else {
    // Listen for restart when the game is over
    if (kb.presses("r")) {
      resetLevel(false);
    }
  }

  // --- BACKGROUNDS ---
  camera.off();
  imageMode(CORNER);
  drawingContext.imageSmoothingEnabled = false;

  for (const layer of bgLayers) {
    const img = layer.img;
    const w = img.width;
    let x = Math.round((-camera.x * layer.speed) % w);

    if (x > 0) x -= w;
    for (let tx = x; tx < VIEWW + w; tx += w) {
      image(img, tx, 0);
    }
  }
  camera.on();

  // --- PIXEL SNAP ---
  const px = player.x, py = player.y;
  const sx = sensor.x, sy = sensor.y;

  player.x = Math.round(player.x);
  player.y = Math.round(player.y);
  sensor.x = Math.round(sensor.x);
  sensor.y = Math.round(sensor.y);

  allSprites.draw();

  player.x = px;
  player.y = py;
  sensor.x = sx;
  sensor.y = sy;
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;
```

```
  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  // Reset HTML UI
  if (uiDiv) {
    uiDiv.show();
    document.getElementById('time-span').innerText = timeLeft + 's';
  }
  if (endDiv) endDiv.style('display', 'none');

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to show you how to add a fun particle effect when the fox finally grabs the treasure?

User: Make the countdown and text larger, and reduce the size of the treasure.

Gemini: Wow, you weren't kidding—that is an absolutely massive treasure chest! Because the game canvas is only 240x192 pixels, any high-resolution image you drop in will completely take over the screen unless we scale it down.

To fix this, we can use the `resize()` function on the `treasureImg` right before we attach it to the sprite. I'll shrink it down to exactly 24 pixels wide so it perfectly matches the size of your level tiles.
I have also bumped up the font size for both the instructions and the countdown timer in the HTML overlay so they are much easier to read.

Here is the updated `sketch.js` code:
JavaScript
```
/*
```

```
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling
Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg, treasureImg;

// UI variables for our HTML overlay
let uiDiv, endDiv;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
```

```
  { c: 5,  r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4,  r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 }
];

// --- TILE MAP ---
let level = [
  "                g                  ", // row  0
  "                                   ", // row  1
  "            LggR                   ", // row  2
  "    LR   LgR         LR            ", // row  3: upper platforms
  "                                   ", // row  4
  "   LgggR      LR   LgR             ", // row  5: mid platforms
  "        LgR           g   LggggR   ", // row  6: walls + low platform
  "            LgR                    ", // row  7
  "                              LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
  "         [d]        gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
```

```
  wallTileRImg = loadImage("assets/wallR.png");

  treasureImg = loadImage("assets/treasure.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- HTML UI OVERLAY (Larger Text!) ---
  uiDiv = createDiv('<div style="font-family: \'Times New Roman\', serif; font-size: 32px; font-weight:
bold;">Find the treasure before time runs out<br><span id="time-span" style="font-size:
42px;">30s</span></div>');
  uiDiv.position(0, 0);
  uiDiv.style('width', '100%');
  uiDiv.style('background', 'rgba(0, 0, 0, 0.6)');
  uiDiv.style('color', 'white');
  uiDiv.style('text-align', 'center');
  uiDiv.style('padding', '15px 0');
  uiDiv.style('pointer-events', 'none');

  endDiv = createDiv('<div id="end-title" style="font-family: \'Times New Roman\', serif; font-size:
64px; font-weight: bold;"></div><div style="font-family: \'Times New Roman\', serif; font-size: 32px;
margin-top: 15px; color: white;">Press R to restart</div>');
  endDiv.position(0, 0);
  endDiv.style('width', '100%');
  endDiv.style('height', '100%');
  endDiv.style('background', 'rgba(0, 0, 0, 0.8)');
  endDiv.style('display', 'none');
  endDiv.style('flex-direction', 'column');
  endDiv.style('justify-content', 'center');
  endDiv.style('align-items', 'center');
  endDiv.style('pointer-events', 'none');

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
```

```
groundDeep.tile = "d";

platformsL = new Group();
platformsL.physics = "static";
platformsL.img = platformTileLImg;
platformsL.tile = "L";

platformsR = new Group();
platformsR.physics = "static";
platformsR.img = platformTileRImg;
platformsR.tile = "R";

wallsL = new Group();
wallsL.physics = "static";
wallsL.img = wallTileLImg;
wallsL.tile = "[";

wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- TREASURE ---
// Scale the massive image down to 24 pixels wide (the '0' tells it to auto-adjust height)
treasureImg.resize(24, 0);

treasure = new Sprite();
treasure.img = treasureImg;
treasure.collider = "static";
treasure.overlaps(player);

resetLevel(false);

// --- GROUND SENSOR ---
sensor = new Sprite();
sensor.x = player.x;
```

```
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  if (gameState === "PLAYING") {
    world.step();

    // --- TIMER LOGIC ---
    if (!timerStarted) {
      if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
        timerStarted = true;
        lastTick = millis();
      }
    } else {
      if (millis() - lastTick >= 1000 && timeLeft > 0) {
        timeLeft--;
        lastTick = millis();
        // Update the HTML timer text
        document.getElementById('time-span').innerText = timeLeft + 's';
      }
    }

    // --- CAMERA ---
    camera.width = VIEWW;
    camera.height = VIEWH;

    let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
    let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

    camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
    camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

    // --- PLAYER CONTROLS ---
    let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);
```

```
// -- ATTACK INPUT --
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4.5;
  jumpSound.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop();
  }
} else {
  if (runSound.isPlaying()) {
```

```
      runSound.stop();
    }
  }

  // --- PLAYER BOUNDS ---
  player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

  // --- GAME END/WIN STATES ---
  if (player.overlaps(treasure)) {
    gameState = "WIN";
    if (runSound.isPlaying()) runSound.stop();

    // Update HTML UI
    uiDiv.hide();
    endDiv.style('display', 'flex');
    document.getElementById('end-title').innerText = "YOU WIN";
    document.getElementById('end-title').style.color = "#00ff00";

  } else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
    if (player.y > LEVELH + TILE_H * 3) {
      fallSound.play();
    }
    gameState = "LOSE";
    if (runSound.isPlaying()) runSound.stop();

    // Update HTML UI
    uiDiv.hide();
    endDiv.style('display', 'flex');
    document.getElementById('end-title').innerText = "YOU LOSE";
    document.getElementById('end-title').style.color = "#ff0000";
  }
} else {
  // Listen for restart when the game is over
  if (kb.presses("r")) {
    resetLevel(false);
  }
}

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
```

```
  camera.on();

  // --- PIXEL SNAP ---
  const px = player.x, py = player.y;
  const sx = sensor.x, sy = sensor.y;

  player.x = Math.round(player.x);
  player.y = Math.round(player.y);
  sensor.x = Math.round(sensor.x);
  sensor.y = Math.round(sensor.y);

  allSprites.draw();

  player.x = px;
  player.y = py;
  sensor.x = sx;
  sensor.y = sy;
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  // Reset HTML UI
  if (uiDiv) {
    uiDiv.show();
    document.getElementById('time-span').innerText = timeLeft + 's';
  }
  if (endDiv) endDiv.style('display', 'none');

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Would you like me to help you add a fun win sound effect to play when the treasure is found?

User: The treasure's scope also needs to be reduced. Currently, the fox has not yet touched the treasure, it is already displayed as a win.

Gemini: Ah, the classic "invisible forcefield" bug!

What is happening here is that p5play is very literal. When we attached your huge treasure image to the sprite, the physics engine immediately drew a massive, invisible collision box around it based on the *original* file size. Even though we visually shrank the image down to 24 pixels, the game's physics engine kept the giant invisible box, so the poor fox bumped into the "edge" of the treasure while still standing on the ground below it.
To fix this, we have to explicitly tell the physics engine to shrink the collision box (the w and h properties) to match our new visual size.
Here is your updated sketch.js code. I forced the treasure's collision box down to 16x16 pixels, meaning the fox has to get right up to the chest to actually trigger the win.
(Replace your entire sketch.js with this code)
JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
    L = platformLC.png       (platform left cap)
    R = platformRC.png       (platform right cap)
    [ = wallL.png            (wall left side)
    ] = wallR.png            (wall right side)
      = empty (no sprite)
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg, treasureImg;

// UI variables for our HTML overlay
```

```
let uiDiv, endDiv;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img, groundTile2Img, platforTileLImg, platforTileRImg, wallTileLImg, wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5,  r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4,  r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 }
];

// --- TILE MAP ---
let level = [
  "                  g                  ", // row  0
  "                                     ", // row  1
  "              LggR                   ", // row  2
  "     LR   LgR         LR             ", // row  3: upper platforms
  "                                     ", // row  4
  "   LgggR       LR   LgR              ", // row  5: mid platforms
  "         LgR           g   LggggR    ", // row  6: walls + low platform
  "              LgR                    ", // row  7
  "                                LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
  "         [d]        gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
```

```
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  treasureImg = loadImage("assets/treasure.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- HTML UI OVERLAY ---
  uiDiv = createDiv('<div style="font-family: \'Times New Roman\', serif; font-size: 32px; font-weight:
bold;">Find the treasure before time runs out<br><span id="time-span" style="font-size:
42px;">30s</span></div>');
  uiDiv.position(0, 0);
  uiDiv.style('width', '100%');
  uiDiv.style('background', 'rgba(0, 0, 0, 0.6)');
```

```
  uiDiv.style('color', 'white');
  uiDiv.style('text-align', 'center');
  uiDiv.style('padding', '15px 0');
  uiDiv.style('pointer-events', 'none');

  endDiv = createDiv('<div id="end-title" style="font-family: \'Times New Roman\', serif; font-size:
64px; font-weight: bold;"></div><div style="font-family: \'Times New Roman\', serif; font-size: 32px;
margin-top: 15px; color: white;">Press R to restart</div>');
  endDiv.position(0, 0);
  endDiv.style('width', '100%');
  endDiv.style('height', '100%');
  endDiv.style('background', 'rgba(0, 0, 0, 0.8)');
  endDiv.style('display', 'none');
  endDiv.style('flex-direction', 'column');
  endDiv.style('justify-content', 'center');
  endDiv.style('align-items', 'center');
  endDiv.style('pointer-events', 'none');

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
  groundDeep.tile = "d";

  platformsL = new Group();
  platformsL.physics = "static";
  platformsL.img = platformTileLImg;
  platformsL.tile = "L";

  platformsR = new Group();
  platformsR.physics = "static";
  platformsR.img = platformTileRImg;
  platformsR.tile = "R";

  wallsL = new Group();
  wallsL.physics = "static";
  wallsL.img = wallTileLImg;
  wallsL.tile = "[";

  wallsR = new Group();
  wallsR.physics = "static";
  wallsR.img = wallTileRImg;
  wallsR.tile = "]";

  new Tiles(level, 0, 0, TILE_W, TILE_H);

  // --- PLAYER ---
  player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
```

```
  player.spriteSheet = playerImg;
  player.rotationLock = true;
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
  player.anis.offset.y = -8;
  player.addAnis(playerAnis);

  player.ani = "idle";
  player.w = 18;
  player.h = 12;
  player.friction = 0;
  player.bounciness = 0;

  // --- TREASURE ---
  treasureImg.resize(24, 0);

  treasure = new Sprite();
  treasure.img = treasureImg;
  // Force the physical collision box to be much smaller than the original image file
  treasure.w = 16;
  treasure.h = 16;
  treasure.collider = "static";
  treasure.overlaps(player);

  resetLevel(false);

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  if (gameState === "PLAYING") {
    world.step();
```

```
  // --- TIMER LOGIC ---
  if (!timerStarted) {
    if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
      timerStarted = true;
      lastTick = millis();
    }
  } else {
    if (millis() - lastTick >= 1000 && timeLeft > 0) {
      timeLeft--;
      lastTick = millis();
      document.getElementById('time-span').innerText = timeLeft + 's';
    }
  }

  // --- CAMERA ---
  camera.width = VIEWW;
  camera.height = VIEWH;

  let targetX = constrain(player.x, VIEWW / 2, LEVELW - VIEWW / 2 - TILE_W / 2);
  let targetY = constrain(player.y, VIEWH / 2 - TILE_H * 2, LEVELH - VIEWH / 2 - TILE_H);

  camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
  camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

  // --- PLAYER CONTROLS ---
  let grounded = sensor.overlapping(ground) || sensor.overlapping(platformsL) ||
sensor.overlapping(platformsR);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play();
  }

  // -- JUMP --
  if (grounded && kb.presses("up")) {
    player.vel.y = -4.5;
    jumpSound.play();
  }

  // --- STATE MACHINE ---
  if (attacking) {
    attackFrameCounter++;
    if (attackFrameCounter > 12) {
      attacking = false;
      attackFrameCounter = 0;
    }
  } else if (!grounded) {
```

```
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop();
  }
} else {
  if (runSound.isPlaying()) {
    runSound.stop();
  }
}

// --- PLAYER BOUNDS ---
player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

// --- GAME END/WIN STATES ---
if (player.overlaps(treasure)) {
  gameState = "WIN";
  if (runSound.isPlaying()) runSound.stop();

  // Update HTML UI
  uiDiv.hide();
  endDiv.style('display', 'flex');
  document.getElementById('end-title').innerText = "YOU WIN";
  document.getElementById('end-title').style.color = "#00ff00";

} else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
  if (player.y > LEVELH + TILE_H * 3) {
    fallSound.play();
  }
  gameState = "LOSE";
  if (runSound.isPlaying()) runSound.stop();
```

```
      // Update HTML UI
      uiDiv.hide();
      endDiv.style('display', 'flex');
      document.getElementById('end-title').innerText = "YOU LOSE";
      document.getElementById('end-title').style.color = "#ff0000";
    }
  } else {
    // Listen for restart when the game is over
    if (kb.presses("r")) {
      resetLevel(false);
    }
  }

  // --- BACKGROUNDS ---
  camera.off();
  imageMode(CORNER);
  drawingContext.imageSmoothingEnabled = false;

  for (const layer of bgLayers) {
    const img = layer.img;
    const w = img.width;
    let x = Math.round((-camera.x * layer.speed) % w);

    if (x > 0) x -= w;
    for (let tx = x; tx < VIEWW + w; tx += w) {
      image(img, tx, 0);
    }
  }
  camera.on();

  // --- PIXEL SNAP ---
  const px = player.x, py = player.y;
  const sx = sensor.x, sy = sensor.y;

  player.x = Math.round(player.x);
  player.y = Math.round(player.y);
  sensor.x = Math.round(sensor.x);
  sensor.y = Math.round(sensor.y);

  allSprites.draw();

  player.x = px;
  player.y = py;
  sensor.x = sx;
  sensor.y = sy;
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;
```

```
  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  // Reset HTML UI
  if (uiDiv) {
    uiDiv.show();
    document.getElementById('time-span').innerText = timeLeft + 's';
  }
  if (endDiv) endDiv.style('display', 'none');

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(1, Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight /
VIEWH)));
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

If you ever run into a collision issue like this again, a handy trick during development is adding `treasure.debug = true;` to temporarily see the exact physical boundaries drawn around your sprites.

Would you like me to show you how to add a fun victory tune when the player successfully grabs the treasure?